

Answer key and marking scheme - not for distribution.

Time: 1 hour 45 minutes

Maximum Marks: 25

Section A - Spreadsheet (Calc)

10 marks

Scenario

GreenLeaf Organics Pvt. Ltd. is a retail chain selling organic products across four regions of the country - North, South, East and West. The company maintains a product master list (product code, name, category and unit price) on one sheet, and a sales register of every order received during January–February 2026 on another sheet. However, the sales register records **only the product code** - it does not repeat the product name, category or price.

The Sales Manager has asked you, the data assistant, to complete the sales register, calculate the revenue figures, and prepare a summary of regional and category-wise performance for the monthly review meeting.

Create a new spreadsheet with two sheets named exactly **Product_Master** and **Sales_Data**, and enter the two tables below.

Sheet: Product_Master

	A	B	C	D
1	Product Code	Product Name	Category	Unit Price
2	P001	Green Tea Pack	Beverages	250
3	P002	Cold Pressed Juice	Beverages	180
4	P003	Organic Honey	Grocery	450
5	P004	Almond Butter	Grocery	620
6	P005	Herbal Shampoo	Personal Care	320
7	P006	Aloe Face Wash	Personal Care	210
8	P007	Quinoa 1 kg	Grocery	540
9	P008	Ginger Ale	Beverages	150

Sheet: Sales_Data

Columns **F**, **G**, **H** and **I** are shaded because they are to be left **blank** when you enter the data - you will fill them using formulas.

	A	B	C	D	E	F	G	H	I
1	Order ID	Order Date	Product Code	Region	Qty	Product Name	Category	Unit Price	Total Amount
2	S01	05-01-2026	P001	North	12				
3	S02	07-01-2026	P003	South	8				
4	S03	09-01-2026	P002	North	20				
5	S04	11-01-2026	P005	East	15				
6	S05	14-01-2026	P004	West	6				
7	S06	16-01-2026	P001	South	10				
8	S07	18-01-2026	P006	North	25				
9	S08	21-01-2026	P007	East	9				
10	S09	23-01-2026	P008	North	30				
11	S10	25-01-2026	P003	West	14				
12	S11	28-01-2026	P002	South	18				
13	S12	30-01-2026	P005	North	11				
14	S13	02-02-2026	P007	South	7				
15	S14	04-02-2026	P004	East	5				
16	S15	06-02-2026	P006	West	22				

Question 1 - Completing and summarising the sales register**[10 marks]****1 (a) Looking up the product details****[2 marks]**

The sales register stores only the product code. On the sheet **Sales_Data**, use the **XLOOKUP** function to bring in the missing details from the sheet **Product_Master** for every order in rows 2 to 16:

- (i) In column **F** - the **Product Name** corresponding to the product code in column **C**.
- (ii) In column **G** - the **Category** of that product.
- (iii) In column **H** - the **Unit Price** of that product.

Condition: the formula must be written once in row 2 and copied down to row 16.

Answer.

In row 2, then select F2:H2 and copy down to row 16:

- (i) F2 =XLOOKUP(\$C2,Product_Master.\$A\$2:\$A\$9,Product_Master.\$B\$2:\$B\$9)
- (ii) G2 =XLOOKUP(\$C2,Product_Master.\$A\$2:\$A\$9,Product_Master.\$C\$2:\$C\$9)
- (iii) H2 =XLOOKUP(\$C2,Product_Master.\$A\$2:\$A\$9,Product_Master.\$D\$2:\$D\$9)

Accept **VLOOKUP** equivalents. The lookup ranges must be absolute (\$) or the formula breaks as it is copied down - **1 mark** for a correct **XLOOKUP**, **1 mark** for all three columns filled to row 16.

1 (b) Order amounts and totals**[2 marks]**

- (i) In column **I** (Total Amount), write a formula to calculate the amount of each order as **Quantity × Unit Price**.
- (ii) In cell **I18**, calculate the **grand total** of all 15 orders.

- (iii) In cell **I19**, calculate the **average order value**.
- (iv) In cell **I20**, display the value of the **highest single order**.

Format the values in column I and in cells I18:I20 as **Currency (Rs.)** with 2 decimal places.

Answer.

- (i) I2 =E2*H2 copied down to row 16
- (ii) I18 =SUM(I2:I16) → **Rs. 60,390.00**
- (iii) I19 =AVERAGE(I2:I16) → **Rs. 4,026.00**
- (iv) I20 =MAX(I2:I16) → **Rs. 6,300.00** (order S10, Organic Honey)

1 mark for the four formulas, **1 mark** for the currency formatting with 2 decimals. Accept a formula that reaches the unit price another way — a nested =E2*XLOOKUP(C2,...) rather than =E2*H2 — provided the amount is right; column H was only ever a convenience.

1 (c) Category and region figures

[2 marks]

Prepare the answers in the cells indicated, using the **SUMIFS** function only:

- (i) Total sales amount of the **Beverages** category in the **North** region - cell **L3**
- (ii) Total sales amount of the **Grocery** category in the **West** region - cell **L4**
- (iii) Total sales amount of orders in the **East** region where Product Name is **Herbal Shampoo** - cell **L5**

Enter a suitable label for each value in the corresponding cells - **K3**, **K4** and **K5**.

Answer.

- (i) L3 label in K3, e.g. “Beverages sales in North”
=SUMIFS(\$I\$2:\$I\$16,\$G\$2:\$G\$16,"Beverages",\$D\$2:\$D\$16,"North") → **Rs. 11,100.00**
- (ii) L4 label in K4, e.g. “Grocery sales in West”
=SUMIFS(\$I\$2:\$I\$16,\$G\$2:\$G\$16,"Grocery",\$D\$2:\$D\$16,"West") → **Rs. 10,020.00**
- (iii) L5 label in K5, e.g. “Herbal Shampoo sales in East”
=SUMIFS(\$I\$2:\$I\$16,\$D\$2:\$D\$16,"East",\$F\$2:\$F\$16,"Herbal Shampoo") → **Rs. 4,800.00**

2 marks for all three correct; **1 mark** for one or two. Note (iii) keys on the product name in column F, not the category - a SUMIFS on category here scores 0 for that item.

1 (d) Region summary and chart

[2 marks]

- (i) In the range **K8:L12**, build a small summary table titled “Region-wise Sales”, listing the four regions in **K9:K12** and their total sales amounts in **L9:L12**. The totals must be calculated using the **SUMIF** function.
- (ii) Using this summary table, insert a **column (bar) chart** on the same sheet.

The chart must have - chart title: “Region-wise Sales - Jan–Feb 2026”; X-axis title: “Region”; Y-axis title: “Total Sales (Rs.)”; and **data labels** displayed on each column.

Answer.

L9 =SUMIF(\$D\$2:\$D\$16,K9,\$I\$2:\$I\$16) copied down L9:L12, against the region labels in K9:K12, giving:

Region	Total Sales
North	Rs. 19,870.00
South	Rs. 13,120.00
East	Rs. 12,760.00
West	Rs. 14,640.00

1 mark for the SUMIF summary table, **1 mark** for the chart carrying all four required elements (title, both axis titles, data labels). A chart built from a hand-typed table rather than SUMIF scores the chart mark only.

1 (e) Pivot table summary

[2 marks]

Insert a **pivot table** (Insert → Pivot Table) using the range **A1:I16** of Sales_Data as the source data. Place the pivot table on a **new sheet** named Pivot_Summary, with the following layout:

- (i) **Row fields** - Region
- (ii) **Column fields** - Category
- (iii) **Data / Values** - Sum of Total Amount

The pivot table must display **row totals** and **column totals**. Finally, sort the pivot table so that the region with the **highest** total sales appears first, and state that region's name in a cell below the pivot table.

Answer.

The completed Pivot_Summary:

Sum of Total Amount	Beverages	Grocery	Personal Care	Total
North	Rs. 11,100.00	-	Rs. 8,770.00	Rs. 19,870.00
South	Rs. 5,740.00	Rs. 7,380.00	-	Rs. 13,120.00
East	-	Rs. 7,960.00	Rs. 4,800.00	Rs. 12,760.00
West	-	Rs. 10,020.00	Rs. 4,620.00	Rs. 14,640.00
Total	Rs. 16,840.00	Rs. 25,360.00	Rs. 18,190.00	Rs. 60,390.00

Sorted by total descending, **North** appears first (Rs. 19,870.00) - that is the name to be stated below the table. **1 mark** for a correct pivot with both margins, **1 mark** for the sort plus the region named. Empty cells (e.g. North/Grocery) are correct - there are no such orders.

- End of Section A -

Section B - Scratch Programming

15 marks

Scenario

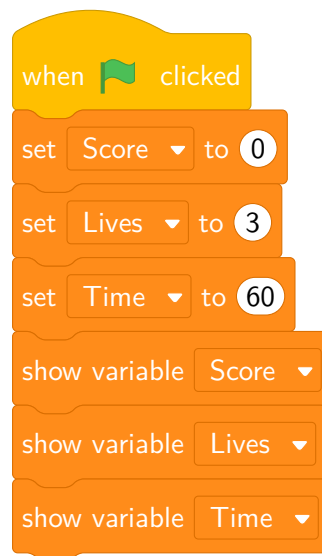
Your school is organising a **Health and Nutrition Week**. The organisers want a small computer game for the exhibition stall that encourages children to collect healthy fruit and avoid the garden pests.

You have been asked to build this game in Scratch. It is called **Apple Catcher**. A bowl at the bottom of the stage is moved left and right by the player. Apples fall from the top of the stage at random positions - catching an apple earns a point. Ladybugs also fall - touching a ladybug costs a life. The game runs for a fixed time or until all lives are lost, after which the final score is displayed.

2 (a) Setting up the project

[3 marks]

- (i) Choose a suitable **backdrop** for the stage (for example a garden, park or sky).
- (ii) Create **three sprites** and rename them exactly as **Bowl**, **Apple** and **Ladybug2**. Resize each sprite so that it is clearly visible but not larger than about one-fifth of the stage.
- (iii) Create **three variables** named **Score**, **Lives** and **Time**, and make all three visible on the stage.
- (iv) When the green flag is clicked, the game must **initialise** the variables to Score = 0, Lives = 3, Time = 60.



Any sprite (or the Stage): initialise the three variables

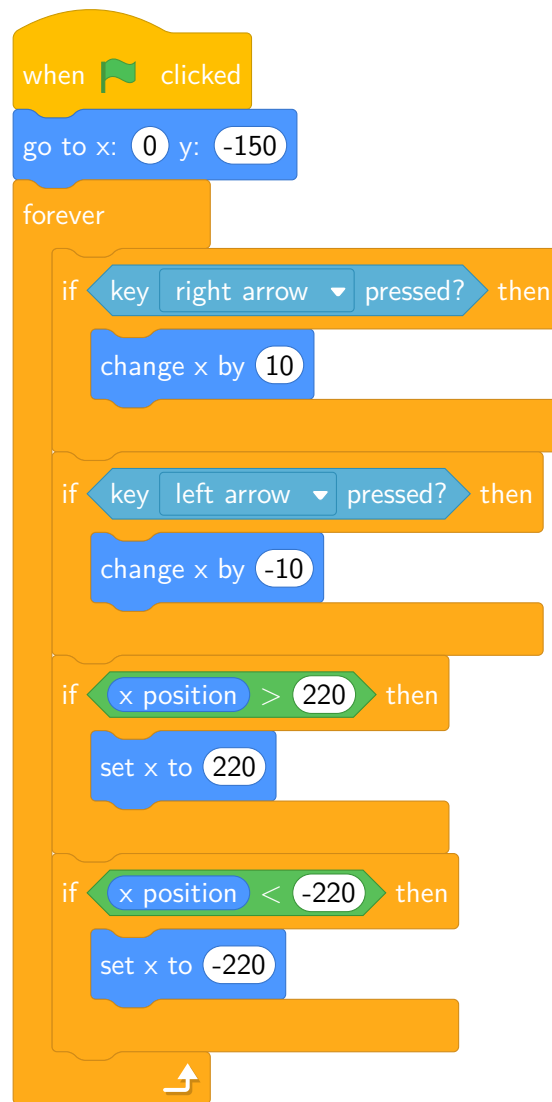
1 mark backdrop and three correctly named, sensibly sized sprites; **1 mark** the three variables created and shown on stage; **1 mark** initialisation under **when green flag clicked**. The initialisation may sit on any sprite or the Stage.

2 (b) Controlling the bowl

[3 marks]

Write the script for the **Bowl** sprite so that:

- (i) It starts at the **bottom centre** of the stage every time the game begins.
- (ii) Pressing the **right arrow** key moves it to the right, and the **left arrow** key moves it to the left. The movement must be **continuous** while the key is held down.
- (iii) The bowl must **not disappear off the edge** of the stage.



Bowl: start at bottom centre, move on the arrow keys, stay on stage

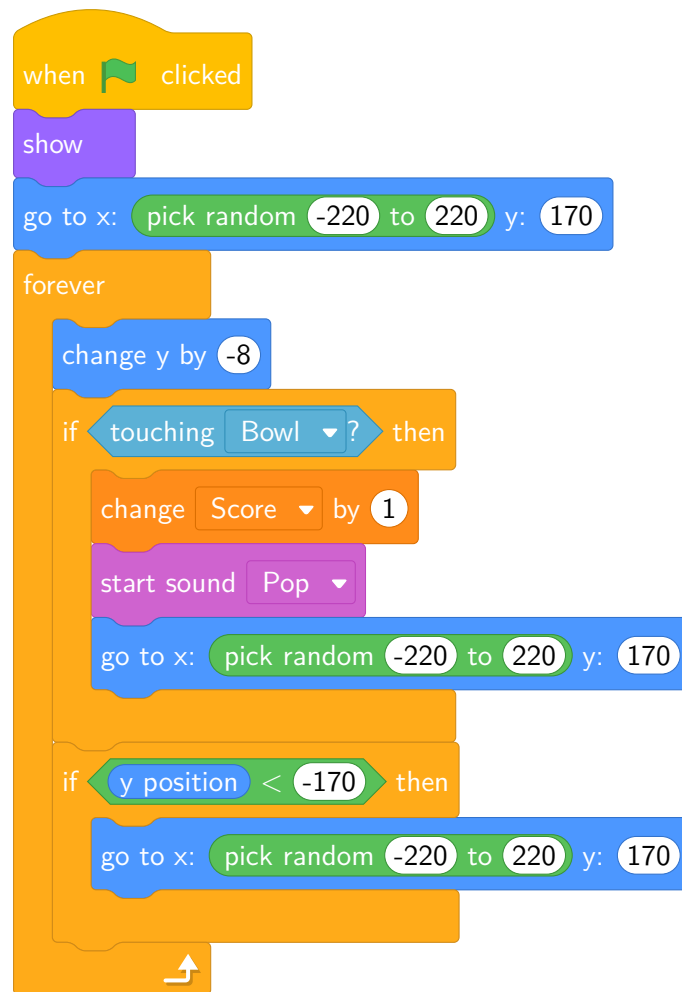
1 mark start position inside a `when green flag clicked` script; **1 mark** both arrow keys read inside a `forever` loop (a `when key pressed` hat also gives continuous movement - accept it); **1 mark** an edge guard. Any working guard scores: the `if x position` clamps shown here, or `if on edge, bounce`.

2 (c) Falling apples and scoring

[4 marks]

Write the script for the `Apple` sprite so that:

- It appears at the **top** of the stage at a **random horizontal position**, using the `pick random` block.
- It **falls downwards** continuously.
- If it **touches the Bowl**, the score increases by 1, a **sound** is played, and the apple returns to the top at a new random position.
- If it reaches the **bottom** of the stage without being caught, it simply returns to the top at a new random position.



Apple: fall from a random x; caught scores a point, missed just resets

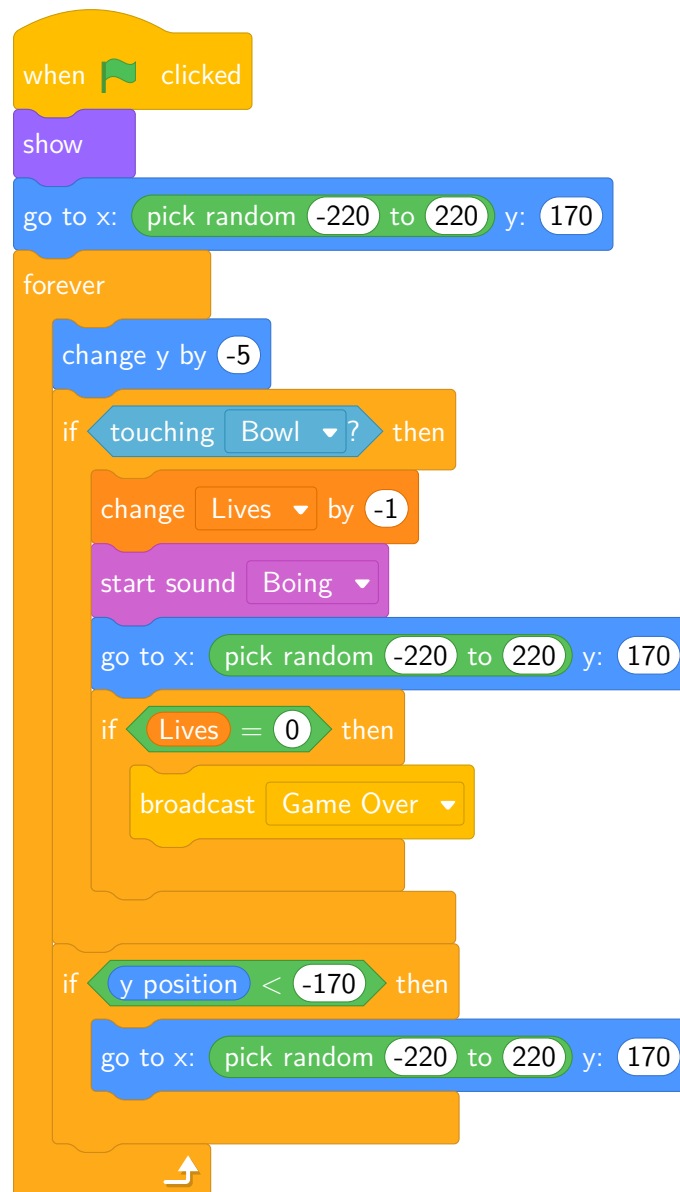
1 mark per numbered requirement. The random spawn must use `pick random` with a sensible x range; falling must be a `change y by` (negative) inside `forever`.

2 (d) The ladybug and losing lives

[3 marks]

Write the script for the `Ladybug2` sprite so that:

- It behaves like the apple - falling from a random position at the top - but moves at a **noticeably different speed**.
- If it touches the `Bowl`, Lives decreases by 1 and a **different sound** is played.
- When Lives reaches 0, the message `Game Over` is **broadcast**.



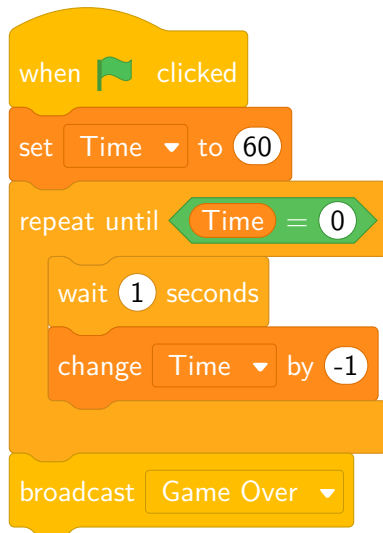
Ladybug2: same fall at a slower speed; a hit costs a life

1 mark falling ladybug at a different step size from the apple; **1 mark** life lost with a different sound; **1 mark** the broadcast when Lives hits 0. Testing `Lives = 0` or `Lives < 1` both score.

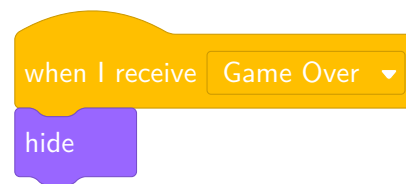
2 (e) Timer and game over

[2 marks]

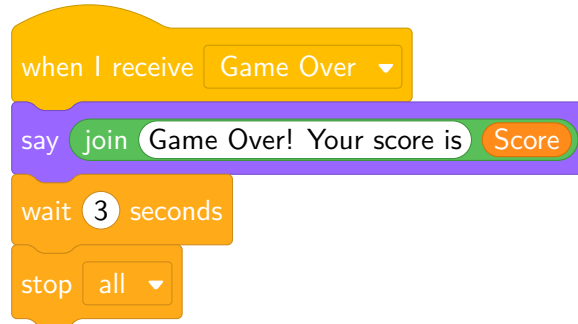
- Write a script that reduces `Time` by 1 every second. When Time reaches 0, the message `Game Over` must be broadcast.
- On receiving `Game Over`: the Apple and the Ladybug2 must **hide**, a message showing the **final score** must be displayed on the stage (for example “Game Over! Your score is 12”), and the whole program must then **stop**.



Timer: set the clock, one second per tick, then end the game



Apple and Ladybug2: leave the stage



Bowl (or Stage): show the final score, then stop everything

1 mark the countdown broadcasting at 0; **1 mark** the receivers - both sprites hidden, the joined score message shown, **stop all** last. The **when I receive** hide script goes on both the Apple and the Ladybug2. Note the countdown keeps running after a lives-out finish, so **Game Over** can be broadcast twice - harmless here, and not penalised.

The model script sets **Time** itself before counting down. Both hats start on the green flag and Scratch fixes no order between them, so a countdown that only reads **Time** can end the game instantly on a second run, when **Time** is still 0 from the last one. **Do not require this** — a student whose timer relies on the setup script scores full marks; it is noted for whoever demonstrates the game twice.

- End of Section B -

- End of Question Paper -